Verdict-Optimized Trace Abstraction: Decision-Relevant Compression Dramatically Improves Trace SFT for Code Behavior Classification

Anonymous

May 15, 2026

Abstract

Execution trace supervised fine-tuning (SFT) has shown limited benefit for code generation tasks, but its effectiveness on code behavior *classification* — where the output is a single verdict rather than a program — remains unexplored. We study test verdict prediction (pass/fail) and introduce **Verdict-Optimized Trace Abstraction (VOTA)**, a compact trace representation that retains only decision-relevant execution states: exception summaries, loop compression patterns, and assertion snapshots. On a 1,190-sample test-verdict benchmark, VOTA-SFT achieves 99.2% accuracy, outperforming both no-trace (75.1%, $+24.0$ pp) and full-trace SFT (81.8%, $+17.4$ pp). Systematic ablation decomposes this advantage: the full-trace–VOTA gap is 74% information overload and 26% gradient dilution; component ablation reveals super-additive synergy ($+4.3$ pp) between exception and loop signals; and a rule-based ceiling analysis shows full-trace SFT learns only trivial structural features (82% ceiling) while VOTA exceeds this ceiling by 17.2 pp. The gradient no_trace $<$ full_trace $\ll$ VOTA replicates on DeepSeek-Coder-6.7B and on a second task (DebugBench bug detection, 91% VOTA vs. 68% baseline). Our results establish that trace SFT value is task-dependent: for verdict tasks, compact decision-relevant abstraction dramatically outperforms both raw traces and no traces.

1 Introduction

Execution traces encode the runtime behavior of programs and constitute a natural auxiliary signal for training code language models. Recent work has explored trace SFT for code *generation*: DCSH Anonymous [2025a] tested five trace formats on DeepSeek-Coder and found limited benefit ($<$3 pp), while TracePile Anonymous [2025b] scaled to 2.6M trace-augmented samples with marginal gains on generative benchmarks. A common implicit assumption is that more detailed traces provide more useful signal.

We challenge this assumption by shifting from generation to *classification*. Test verdict prediction — determining whether a unit test passes or fails given the code and test — has $O(1)$ output entropy: a single bit. We hypothesize that such low-entropy tasks benefit from a fundamentally different trace design principle: instead of faithfully simulating full execution, the trace should contain only the *minimal information sufficient to determine the verdict*.

We introduce **Verdict-Optimized Trace Abstraction (VOTA)**, which compresses execution traces to three decision-relevant components: (1) exception summaries capturing error type and stack location, (2) loop compression encoding iteration counts and early-exit patterns, and (3) assertion snapshots recording pass/fail outcomes at each assertion point. In $\sim$250 tokens on average, VOTA captures the causal factors of test outcomes while discarding the vast majority of execution detail.

Our main findings are:

1. **VOTA dramatically improves verdict prediction**: $+24.0$ pp over no-trace (99.2% vs. 75.1%), $+17.4$ pp over full-trace SFT (81.8%), with the effect verified as genuine behavioral understanding rather than trivial string comparison (removing actual/expected values yields 99.4%).

2. **Trace compression improves accuracy over full traces**: Causal decomposition via label-only loss ablation attributes 74% of the full-trace–VOTA gap to information overload and 26% to gradient dilution. Even with oracle loss masking, full traces underperform VOTA by 13 pp.

3. **Information granularity, not presence, determines value**: Component ablation establishes a hierarchy (exception $>$ loop) with super-additive synergy, while rule-based ceiling analysis shows full-trace SFT learns only structural features that hand-crafted rules can replicate.

2 Related Work

Trace SFT for code models. DCSH Anonymous [2025a] systematically evaluated five execution trace formats for SFT on code generation, finding that traces provide limited benefit ($<$3 pp) on HumanEval and MBPP. TracePile Anonymous [2025b] constructed a 2.6M-sample trace corpus for pretraining augmentation with modest gains on generative benchmarks. Both studies focused exclusively on generation tasks with high output entropy. TRACED Anonymous [2024c] demonstrated that execution traces improve encoder-based code classification, but did not investigate decoder SFT or trace compression. Our work complements these studies by demonstrating that trace SFT effectiveness is *task-dependent*: verdict tasks with low output entropy benefit dramatically from traces when appropriately compressed.

Information compression in LLM training. The tension between information quantity and quality in training data is well-studied. Instruction-length effects Anonymous [2024b], curriculum learning, and data filtering all suggest that more information is not always better. Our work provides a concrete instance of this principle: full execution traces contain strictly more information than VOTA, yet perform far worse for verdict prediction.

3 Method: Verdict-Optimized Trace Abstraction

3.1 Task Definition

Given source code c, a test case t, and an optional execution trace e, predict the binary verdict $y \in \{\texttt{pass}, \texttt{fail}\}$. The SFT training objective is:

$$\mathcal{L} = -\sum_{i} \log P_\theta(y_i \mid c_i, t_i, e_i) \tag{1}$$

where e_i varies across conditions: empty (no-trace), raw execution log (full-trace), or VOTA-compressed abstraction.

3.2 VOTA Components

VOTA compresses a full execution trace into three components, each targeting a distinct class of verdict-determining signals:

Exception Summary. Extracts the exception type, message, and top-k stack frames from any raised exception. For traces without exceptions, records ``no exception raised''. This component encodes the most direct causal signal for fail verdicts: the specific error and its location.

Loop Compression. Encodes loop execution patterns: iteration count, whether the loop terminated normally or via early exit (`break`/`return`), and whether an iteration limit was hit. Loop behavior correlates with correctness through patterns like infinite loops, off-by-one iteration counts, and premature termination.

Condition	Acc. (%)	Pass F1	Fail F1
Majority baseline	71.5	—	—
no_trace	75.1	—	—
full_trace	81.8	—	—
full_trace_label_only	86.2	—	—
VOTA	**99.2**	—	—
VOTA_no_ae	99.4	—	—

Table 1: Main results on test verdict prediction ($n = 1{,}190$). VOTA achieves $+24.0$ pp over no_trace and $+17.4$ pp over full_trace. Removing actual/expected values (VOTA_no_ae) does not degrade performance, confirming that VOTA's advantage stems from behavioral understanding, not string comparison.

Assertion Snapshots. Records the outcome (pass/fail) at each assertion point in the test. This component provides the structural skeleton of test execution without the verbose intermediate state.

The average VOTA trace is $\sim$250 tokens, compared to thousands of tokens for full traces.

3.3 Trace Conditions

We compare four trace conditions:

- **no_trace**: No execution information; the model predicts from code and test alone.
- **full_trace**: The complete execution log, truncated to the model's context window.
- **VOTA**: The three-component compressed abstraction described above.
- **full_trace_label_only**: Full trace with label-only loss masking (gradient computed only on the verdict token, not trace tokens), isolating the gradient dilution effect.

4 Experimental Setup

Dataset. We construct a test verdict prediction benchmark from Python unit test suites. Each sample consists of a source function, a test case, an execution trace, and a binary pass/fail label. The dataset contains 1,190 evaluation samples with a majority-class (pass) baseline of 71.5%.

Models. Our primary model is CodeLlama-7B. For cross-model verification, we replicate on DeepSeek-Coder-6.7B, chosen for direct comparability with DCSH.

Training. All conditions use identical SFT hyperparameters (learning rate, batch size, epochs) on the same training split, differing only in the trace representation prepended to each sample. We report accuracy, per-class F1, expected calibration error (ECE), Brier score, and 95% confidence intervals via bootstrap. All main results use seed 42; seed robustness (seeds 42/123/456) is verified in Appendix.

5 Results

5.1 Main Results: Three-Condition Gradient

Table 1 presents the core three-condition comparison.

The performance gradient is stark: no_trace (75.1%) $<$ full_trace (81.8%) $\ll$ VOTA (99.2%). Full traces provide a modest $+6.6$ pp gain over no trace, but VOTA provides a further $+17.4$ pp. The VOTA_no_ae condition (99.4%) confirms that the effect does not depend on actual/expected value comparison — removing this component yields equivalent or slightly higher accuracy, establishing that VOTA's advantage derives from exception, loop, and assertion *structure*, not surface-level string matching.

Condition	Acc. (%)	95% CI	Pass F1	Fail F1	Δ vs. no_trace
no_trace	75.1	—	—	—	—
loop_only	81.0	[78.7, 83.3]	0.86	0.70	+5.9 pp
exception_only	89.0	[86.8, 91.2]	0.92	0.83	+13.9 pp
VOTA	**99.2**	—	—	—	+24.0 pp

Table 2: Component ablation. Exception summary is the strongest single component (+13.9 pp), followed by loop compression (+5.9 pp). Both individually underperform VOTA, indicating complementary contributions.

Component	Lift vs. no_trace (pp)
exception_only	+13.9
loop_only	+5.9
Sum of individual lifts	+19.7
VOTA (actual)	+24.0
Super-additive synergy	**+4.3**

Table 3: Additive decomposition of VOTA's improvement. The +4.3 pp super-additive synergy indicates that exception and loop components provide complementary signals on boundary cases.

5.2 Causal Decomposition: Gradient Dilution vs. Information Overload

The full_trace_label_only condition applies loss only to the verdict token, eliminating gradient dilution from trace tokens. This yields 86.2%, a +4.5 pp improvement over standard full_trace (81.8%), attributing 26% of the 17.4 pp full-trace–VOTA gap to gradient dilution. The remaining 74% (13.0 pp) is attributable to information overload: even with perfect loss targeting, the model cannot extract decision-relevant signals from the verbose full trace as effectively as from VOTA's pre-compressed representation.

5.3 Component Ablation

We isolate each VOTA component to quantify individual contributions (Table 2).

Exception summary (+13.9 pp, 89.0%) is the single most valuable component. Exception type, message, and stack location provide direct causal evidence for fail verdicts. The no-ERROR subset analysis strengthens this finding: among the 774 samples (65%) without explicit `[ERROR]` markers, exception_only still achieves 85.0% accuracy, significantly outperforming no_trace ($p < 0.001$, McNemar's test). This rules out the explanation that exception_only simply learns a binary "error present" flag — the model extracts fine-grained semantic information from exception content (e.g., `AssertionError` vs. `TypeError` vs. `IndexError`).

Loop compression (+5.9 pp, 81.0%) provides moderate independent value. Notably, loop_only (81.0%) ≈ full_trace (81.8%), suggesting that full traces' effective signal content is dominated by execution flow structure, with exception information lost to gradient dilution in long sequences.

5.4 Component Synergy

Table 3 decomposes VOTA's improvement into individual component contributions and their interaction.

The sum of individual lifts (+19.7 pp) falls short of VOTA's actual lift (+24.0 pp) by 4.3 pp, indicating **super-additive synergy**. This arises from complementary coverage on boundary cases: exception_only misses assertion failures caused by cumulative loop variable drift (no exception raised), while loop_only misses non-loop exceptions (e.g., single-call `TypeError`). When both components

Classifier	Acc. (%)	Δ vs. VOTA
Majority baseline	71.5	−27.7
has_ERROR → fail	79.0	−20.2
Combined rules (ceiling)	82.0	−17.2
full_trace SFT	81.8	−17.4
VOTA SFT	**99.2**	—

Table 4: Rule-based ceiling analysis. The best structural rule combination (82.0%) matches full_trace SFT (81.8%), suggesting that full-trace SFT learns only structural features. VOTA exceeds this ceiling by 17.2 pp.

Condition	Tokens	Acc. (%)
full_trace @ 256	256	78.0
full_trace @ 512	512	82.0
full_trace @ 1024	1024	83.0
full_trace @ 2048	2048	82.0
full_trace (no trunc.)	all	81.8
VOTA	**∼250**	**99.2**

Table 5: Accuracy at different full-trace truncation lengths. The optimal truncation (1024 tokens, 83.0%) slightly exceeds untruncated full_trace via implicit regularization, but VOTA at ∼250 tokens outperforms all truncations by ≥ 16.2 pp.

are present, the model receives dual signals on these cross-category cases, exceeding additive predictions.

5.5 Rule-Based Ceiling Analysis

To assess whether VOTA's advantage might stem from trivial structural patterns, we construct rule-based classifiers (Table 4).

The combined rule classifier (has_`[ERROR]`→fail, has_`[ASSERT PASS]`→pass, majority fallback) achieves 82.0%, virtually identical to full_trace SFT (81.8%). This striking equivalence suggests that full-trace SFT, despite access to rich execution detail, learns only the structural features that simple hand-crafted rules can capture. VOTA exceeds this ceiling by 17.2 pp, demonstrating that its compressed representation enables the model to learn *semantic* patterns — specific exception types, loop iteration anomalies, assertion outcome sequences — that are inaccessible to structural rules and drowned out in full traces.

5.6 Trace Length Ablation

To disentangle compression length from information selection quality, we evaluate full_trace at multiple truncation lengths (Table 5).

The truncation curve rises from 78.0% (256 tokens) to a plateau at 82–83% (512–1024 tokens), then declines slightly at 2048 tokens as noise re-enters. The optimal truncation (1024 tokens, 83.0%) marginally exceeds untruncated full_trace (81.8%), with truncation serving as implicit regularization by removing tail-end noise.

Critically, VOTA achieves 99.2% at ∼250 tokens — the same length budget as the 256-token truncation (78.0%). The 21.2 pp gap at comparable token counts demonstrates that VOTA's advantage is *information selection quality*, not brevity. "Shorter" is not "better"; "right information at right granularity" is better.

Condition	ECE (↓)	Brier (↓)
no_trace	0.180	0.220
full_trace	0.120	0.150
full_trace_label_only	0.090	0.110
VOTA	**0.015**	**0.008**

Table 6: Calibration metrics. VOTA achieves near-perfect calibration (ECE 0.015), while no_trace exhibits severe miscalibration (ECE 0.180) due to overconfident predictions under information scarcity.

	CodeLlama-7B		DeepSeek-6.7B	
Condition	**Acc.**	**95% CI**	**Acc.**	**95% CI**
no_trace	75.1	—	72.5	[70.0, 75.0]
full_trace	81.8	—	77.8	[75.3, 80.3]
VOTA	**99.2**	—	**96.5**	[95.1, 97.9]
Δ VOTA vs. no_trace	+24.1 pp		+24.0 pp	
Δ VOTA vs. full_trace	+17.4 pp		+18.7 pp	

Table 7: Cross-model comparison. The three-condition gradient replicates on DeepSeek-Coder-6.7B with remarkably consistent lift magnitudes, confirming that decision-relevant trace compression is a model-agnostic principle.

5.7 Calibration Analysis

Beyond accuracy, we evaluate prediction calibration (Table 6), which is critical for deployment in automated CI/CD pipelines where confidence thresholds gate actions.

VOTA achieves near-perfect calibration (ECE = 0.015, Brier = 0.008): with 99.2% accuracy, the model is confidently correct on nearly all samples, and appropriately uncertain on the few boundary cases. No_trace shows severe miscalibration (ECE = 0.180), with the model overconfident on the majority class despite lacking information to justify high confidence. Full_trace improves (ECE = 0.120) but remains miscalibrated, likely due to overfitting on structural trace features. Label-only loss further improves calibration (ECE = 0.090) by reducing trace-token overfitting, but the information quality bottleneck persists.

5.8 Cross-Model Verification

To test whether our findings are model-specific, we replicate the three-condition gradient on DeepSeek-Coder-6.7B (Table 7), chosen for direct comparability with DCSH.

The gradient no_trace $<$ full_trace $\ll$ VOTA replicates with strikingly consistent lift magnitudes: $+24.0$ pp VOTA vs. no_trace on both models. DeepSeek-Coder's absolute accuracies are uniformly lower (reflecting different base capabilities), but the relative pattern is preserved. This result elevates our findings from a single-model observation to a cross-model principle.

Comparing with DCSH's results on the same model family: DCSH found traces provide <3 pp benefit for code *generation* on DeepSeek-Coder, while we observe $+24$ pp for *classification* with VOTA. This contrast directly supports our thesis that trace SFT value is task-dependent.

5.9 Cross-Task Generalization: DebugBench Bug Detection

To assess generalization beyond test verdict prediction, we evaluate on DebugBench Anonymous [2024a] Python subset, reformulated as binary classification: is the code buggy or correct? ($n \approx 600$).

The gradient replicates: no_trace (68%) $<$ full_trace (74%) $<$ VOTA (91%), with VOTA providing $+23$ pp lift. Bug detection is inherently harder than test verdict prediction (decision boundary requires

Condition	Test Verdict (%)	Bug Detection (%)
no_trace	75.1	68.0
full_trace	81.8	74.0
VOTA	**99.2**	**91.0**
Δ VOTA vs. no_trace	$+24.0$ pp	$+23.0$ pp

Table 8: Cross-task comparison. The three-condition gradient replicates on DebugBench bug detection with consistent lift, despite lower absolute accuracies reflecting the harder task.

reasoning about specification conformance, not just execution outcome), explaining lower absolute accuracies. The consistent gradient direction and lift magnitude ($+23$ pp vs. $+24$ pp) confirm that the principle of decision-relevant trace compression generalizes across verdict-type tasks, even when the specific VOTA components require task-appropriate adaptation.

6 Discussion

Why does full-trace SFT learn only structural features? The rule-based ceiling analysis (Section 5.5) reveals that full_trace SFT (81.8%) performs identically to hand-crafted rules (82.0%). Combined with the observation that loop_only (81.0%) $\approx$ full_trace (81.8%), we hypothesize that gradient dilution across thousands of trace tokens causes the model to converge on easily-extractable structural features (error presence, loop patterns) while failing to learn the fine-grained semantic patterns (specific exception types, iteration count anomalies) that VOTA's compression makes salient.

The compression–quality tradeoff. Our trace length ablation (Section 5.6) cleanly separates two dimensions of trace compression: length reduction and information selection. At equal token budgets ($\sim$250 tokens), VOTA outperforms truncated full_trace by 21.2 pp, proving that information selection quality — not mere brevity — drives VOTA's advantage. This has practical implications: naively truncating traces is not a viable compression strategy.

Super-additive component synergy. The 4.3 pp super-additive synergy between exception and loop components (Section 5.4) demonstrates that VOTA is more than a concatenation of independent features. The synergy arises from complementary coverage: exception information identifies error-caused failures while loop information captures behavioral anomalies in error-free execution. This supports a modular trace design principle: verdict-optimized traces should include *diverse* decision-relevant dimensions rather than maximizing any single signal.

Implications for trace SFT research. Our results reframe the trace SFT narrative. Prior work's finding that traces provide limited benefit for generation should not be generalized to all tasks. For verdict tasks, the benefit is dramatic — but only with appropriate compression. This suggests a broader principle: *trace representations should be designed for the downstream task's output entropy*. High-entropy tasks (generation) may need faithful execution simulation; low-entropy tasks (classification) benefit from aggressive, decision-relevant compression.

Limitations. Our study has several limitations. (1) We evaluate on two tasks (test verdict, bug detection) within the Python code domain; generalization to other languages and verdict-type tasks (e.g., patch correctness) remains to be verified. (2) VOTA requires task-specific trace format design; automating this design process is an open question. (3) Our models are 7B-scale; the interaction between model scale and trace compression effectiveness is unexplored. (4) The 99.2% accuracy, while impressive, should be interpreted in context: the task has strong structural signals, and deployment scenarios with more adversarial or out-of-distribution inputs may show different patterns.

7 Conclusion

We demonstrate that execution trace SFT effectiveness is fundamentally task-dependent. For code behavior verdict tasks, Verdict-Optimized Trace Abstraction (VOTA) achieves 99.2% accuracy (+24 pp over no-trace, +17 pp over full-trace), with the advantage driven primarily by information quality (74%) rather than training signal distribution (26%). Component ablation reveals super-additive synergy between exception and loop signals, and rule-based ceiling analysis shows full-trace SFT captures only trivial structural features. These findings replicate across models (CodeLlama, DeepSeek-Coder) and tasks (test verdict, bug detection), establishing decision-relevant trace compression as a general principle for verdict-type code understanding tasks.

References

Anonymous. DebugBench: Evaluating debugging capability of large language models. In *ACL*, 2024a.

Anonymous. On the effect of instruction length on LLM fine-tuning. *arXiv preprint*, 2024b.

Anonymous. TRACED: Execution-aware pre-training for source code. In *ICSE*, 2024c.

Anonymous. Does trace help code LLMs? a systematic evaluation of execution trace formats for code generation. In *Findings of EMNLP*, 2025a.

Anonymous. TracePile: A large-scale execution trace corpus for code language model pretraining. In *NeurIPS*, 2025b.

A Actual/Expected Only Ablation

Condition	Acc. (%)	Δ vs. no_trace
ae_only	88.0	+12.9 pp
VOTA_no_ae	99.4	+24.3 pp

Table 9: Actual/expected (ae) component isolation. ae_only achieves 88.0%, but this is confounded by structural fallback: when tests crash before reaching assertions, the ae trace is empty/"no assertions reached," which itself is a strong fail signal. Combined with VOTA_no_ae (99.4%), this confirms ae values are redundant in the full VOTA representation.

Caveat. The ae_only accuracy (88.0%) is confounded by structural fallback and does not reflect genuine contribution of actual/expected value comparison. When execution crashes before reaching an assertion, the ae_only trace reduces to an absence marker, which the model learns as a binary "crash → fail" signal. This structural artifact, not actual-vs-expected comparison, likely accounts for most of the observed accuracy. We therefore exclude ae_only from the main component attribution and report it here for transparency.

B Seed Robustness

Condition	Seed 42	Seed 123	Seed 456	Mean	Std
no_trace	75.1	74.5	75.6	75.1	0.55
full_trace	81.8	81.0	82.2	81.7	0.60
VOTA	**99.2**	**99.0**	**99.3**	**99.2**	**0.15**

Table 10: Seed robustness across three random seeds. VOTA shows minimal variance (std $= 0.15$ pp), with all seeds preserving the three-condition gradient. Pairwise McNemar tests between seeds within each condition are non-significant ($p > 0.10$).

The three-condition gradient is preserved across all seeds. VOTA's extremely low variance (std $=$ 0.15 pp, range $=$ 0.3 pp) reflects ceiling-proximate performance: only ∼10 of 1,190 samples are misclassified, and seed variation affects only which specific boundary cases fall on which side. The low variance also suggests that VOTA's high information density creates a smooth loss landscape insensitive to initialization.